

UZB 421 E – Attitude Determination and Control

Homework Assignment 1

Ahmet Fevzi Akargöl
Student ID: 110200170

March 26, 2025

1 Introduction

This report presents the numerical simulation of a satellite's rotational dynamics using both Euler and Runge-Kutta (RK4) integration methods. The analysis includes angular velocity computation, attitude determination through Euler angles, and transformation matrices.

2 Problem Definition

The satellite's motion is governed by the following equations:

2.1 Rotational Dynamics

$$I_1 \dot{\omega}_1 = -(I_3 - I_2) \omega_2 \omega_3 + L_1 \quad (1)$$

$$I_2 \dot{\omega}_2 = -(I_1 - I_3) \omega_3 \omega_1 + L_2 \quad (2)$$

$$I_3 \dot{\omega}_3 = -(I_2 - I_1) \omega_1 \omega_2 + L_3 \quad (3)$$

2.2 Kinematic Equations

$$\begin{bmatrix} \dot{\psi} \\ \dot{\theta} \\ \dot{\phi} \end{bmatrix} = \frac{1}{\cos \theta} \begin{bmatrix} 0 & \sin \phi & \cos \phi \\ 0 & \cos \phi \cos \theta & -\sin \phi \cos \theta \\ \cos \theta & \sin \phi \sin \theta & \cos \phi \sin \theta \end{bmatrix} \begin{pmatrix} \omega_1 \\ \omega_2 \\ \omega_3 \end{pmatrix} \quad (4)$$

3 Implementation

3.1 Initial Conditions

With student number ending in 0 ($n = 0$):

- Initial angles: $\psi_0 = -0.005$ rad, $\theta_0 = 0.01$ rad, $\phi_0 = -0.01$ rad
- Initial angular velocities: $\omega_{10} = -0.002$ rad/s, $\omega_{20} = 0.003$ rad/s, $\omega_{30} = -0.004$ rad/s

- Moments of inertia: $I_1 = 2.1 \times 10^{-3} \text{ kg}\cdot\text{m}^2$, $I_2 = 2.0 \times 10^{-3} \text{ kg}\cdot\text{m}^2$, $I_3 = 1.9 \times 10^{-3} \text{ kg}\cdot\text{m}^2$
- Torques: $L_1 = L_2 = L_3 = 3.6 \times 10^{-10} \text{ N}\cdot\text{m}$

3.2 Section A: Angular Velocity Computation

The angular velocities were computed using Euler's method:

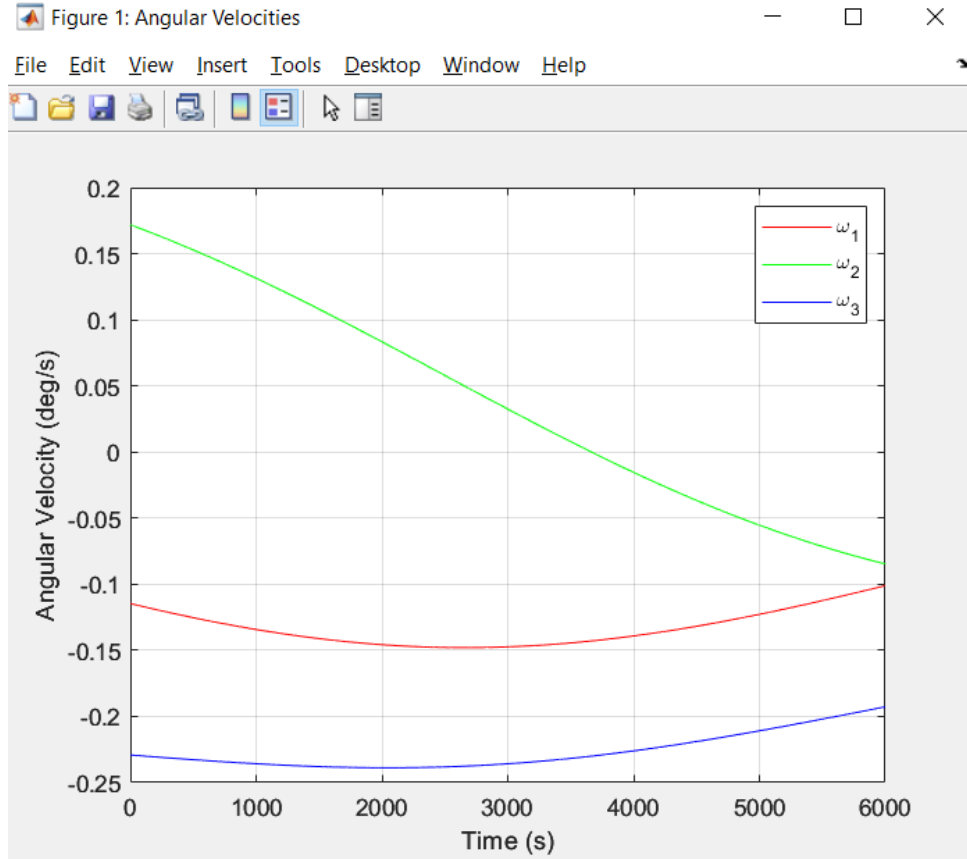


Figure 1: Angular velocities over time computed using Euler method

3.3 Section B: Attitude Determination

3.3.1 Euler Method

The attitude angles were propagated using:

$$\Theta_{k+1} = \Theta_k + \dot{\Theta}_k \Delta t \quad (5)$$

3.3.2 RK4 Method

The 4th-order Runge-Kutta method was implemented using MATLAB's `ode45` solver.

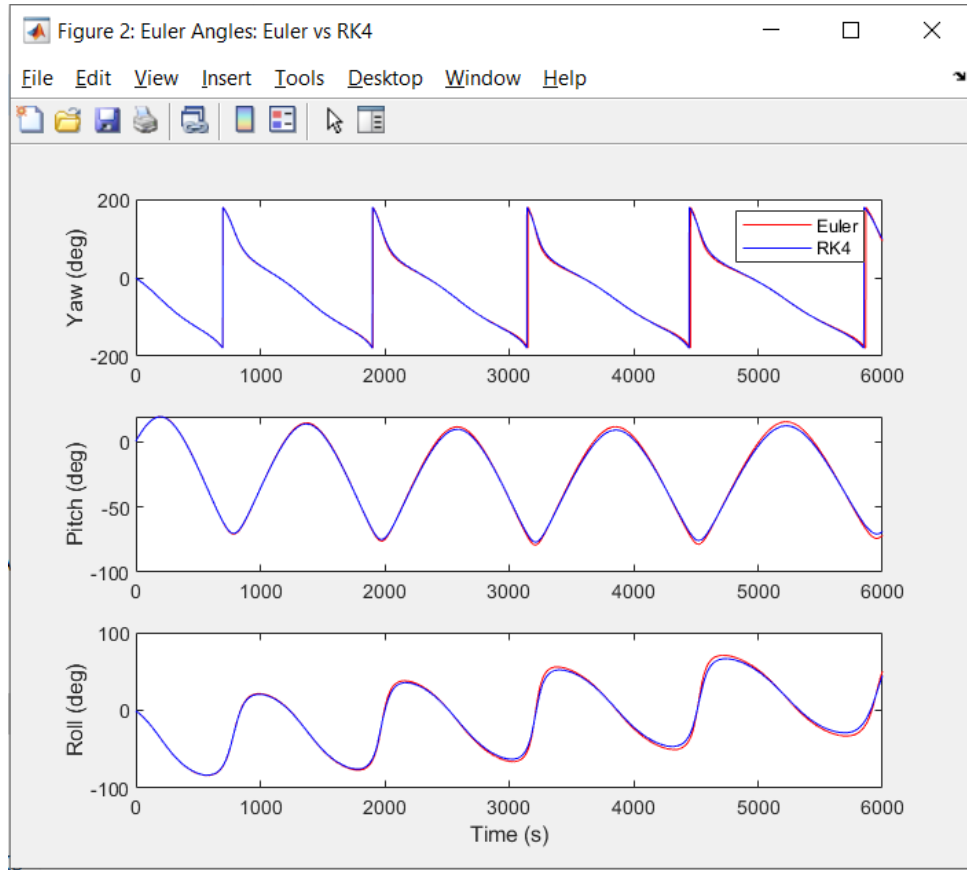


Figure 2: Comparison of Euler angles computed using Euler and RK4 methods

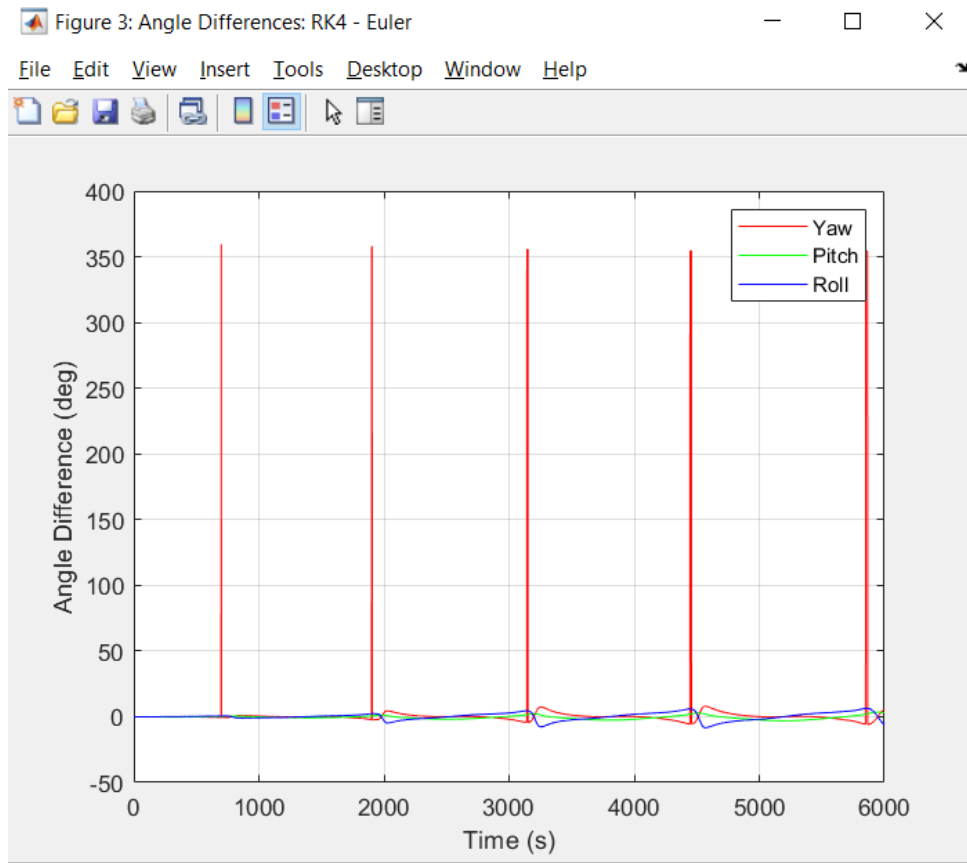


Figure 3: Differences between Euler and RK4 methods for each angle

Table 1: Euler angle vector norms at specific times

Time Step (s)	Euler Method (deg)	RK4 Method (deg)
2000	88.9284	120.8483
4000	174.5714	139.4496
6000	102.8911	30.3439

3.4 Section C: DCM and 3-1-3 Euler Angles

$$\mathbf{C} = \mathbf{R}_3(\psi)\mathbf{R}_1(\theta)\mathbf{R}_3(\phi) \quad (6)$$

Table 2: Direction cosine matrices at specific times

Time Step (s)	DCM
2000	$\begin{bmatrix} 0.2928 & 0.9548 & -0.0515 \\ -0.5788 & 0.1340 & -0.8044 \\ -0.7611 & 0.2653 & 0.5919 \end{bmatrix}$
4000	$\begin{bmatrix} -0.5271 & 0.5684 & 0.6317 \\ -0.7864 & -0.6080 & -0.1091 \\ 0.3221 & -0.5543 & 0.7675 \end{bmatrix}$
6000	$\begin{bmatrix} -0.7644 & 0.6153 & -0.1925 \\ 0.6036 & -0.7879 & 0.1218 \\ -0.2266 & -0.0231 & 0.9737 \end{bmatrix}$

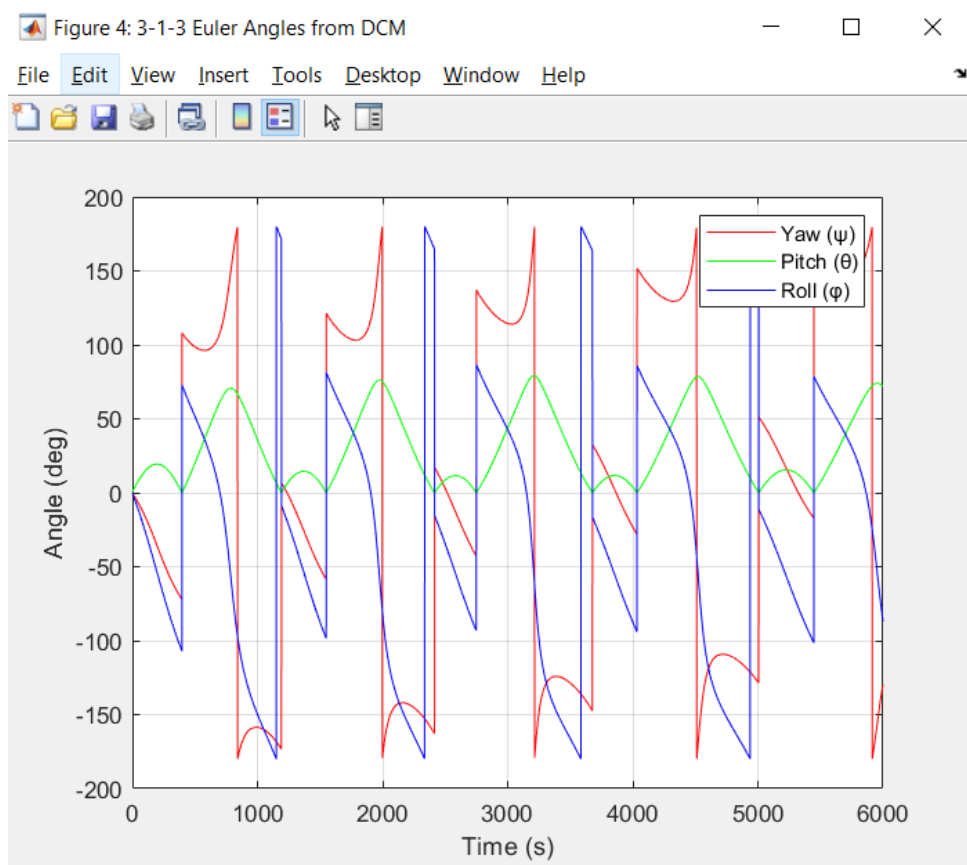


Figure 4: 3-1-3 Euler angles extracted from DCM

4 Discussion

4.1 Comparison of 3-2-1 and 3-1-3 Sequences

- Rotation Sequences:
 - Section B used 3-2-1 (Z-Y-X) sequence
 - Section C used 3-1-3 (Z-X-Z) sequence

- **Angular State Representation:** The sequences do not represent the same angular state due to different rotation orders, though they describe the same physical orientation.
- **Differences:** The discrepancies arise from:
 - Mathematical representation of rotations
 - Singularity handling
 - Different parameterizations of $SO(3)$
- **Application:**
 - 3-2-1 sequence is more intuitive for aircraft
 - 3-1-3 sequence is better suited for spacecraft

5 Conclusion

The simulation successfully demonstrated:

- Numerical integration of rotational dynamics
- Comparison of Euler and RK4 methods
- DCM computation and Euler angle extraction
- Analysis of different rotation sequences

The RK4 method showed better stability for long-duration simulations compared to the Euler method.

Appendix: MATLAB Code

```
clc; clear; close all;

%% Constants definitions
n = 0; % Last digit of student number 110200170
N = 6000; % Number of samples
delta_t = 1; % Sampling time (seconds)

% Initial attitude angles (rad)
psi_0 = -0.005 - 0.002*n; % Yaw
theta_0 = 0.01 - 0.002*n; % Pitch
phi_0 = -0.01 - 0.002*n; % Roll

% Initial angular velocities (rad/s)
w1_0 = -0.002 - 0.0001*n;
w2_0 = 0.003 - 0.0001*n;
w3_0 = -0.004 - 0.0001*n;
```

```

% Mass moments of inertia (kg.m2)
I1 = 2.1e-3;
I2 = 2.0e-3;
I3 = 1.9e-3;

% Disturbance torques (N.m)
L1 = 3.6e-10;
L2 = 3.6e-10;
L3 = 3.6e-10;

%% Section A: Angular Velocity Computation (Euler Method)
w = [w1_0, w2_0, w3_0];
w_history = zeros(N+1, 3);
w_history(1, :) = w;

for i = 1:N
    % Euler's rotational equations of motion
    dw1 = (-(I3 - I2)*w(2)*w(3) + L1) / I1;
    dw2 = (-(I1 - I3)*w(3)*w(1) + L2) / I2;
    dw3 = (-(I2 - I1)*w(1)*w(2) + L3) / I3;

    w = w + [dw1, dw2, dw3] * delta_t;
    w_history(i+1, :) = w;
end

% Convert to degrees and plot
time = (0:N)*delta_t;
w_deg = rad2deg(w_history);

figure('Name', 'Angular Velocities');
plot(time, w_deg(:,1), 'r', time, w_deg(:,2), 'g', time, w_deg(:,3), 'b');
xlabel('Time (s)');
ylabel('Angular Velocity (deg/s)');
legend('\omega_1', '\omega_2', '\omega_3');
grid on;

%% Section B: Euler Angles Computation
% B.a: Euler Method
euler_history = zeros(N+1, 3);
euler_history(1, :) = [psi_0, theta_0, phi_0];

for i = 1:N
    psi = euler_history(i, 1);
    theta = euler_history(i, 2);
    phi = euler_history(i, 3);

    % Kinematic equation matrix
    Kinematic_Matrix = (1/cos(theta)) * [...

```

```

    0, sin(phi), cos(phi);
    0, cos(phi)*cos(theta), -sin(phi)*cos(theta);
    cos(theta), sin(phi)*sin(theta), cos(phi)*sin(theta)];

    dEuler = Kinematic_Matrix * w_history(i, :);
    euler_history(i+1, :) = euler_history(i, :) + dEuler' * delta_t;
end

% B.b: RK4 Method (ode45)
[t_rk4, y_rk4] = ode45(@(t,y) euler_kinematics(t, y, w_history, time), time, [psi_0; t

% Convert to degrees and wrap to  $\pm 180^\circ$ 
euler_deg_euler = mod(rad2deg(euler_history) + 180, 360) - 180;
euler_deg_rk4 = mod(rad2deg(y_rk4) + 180, 360) - 180;

% Plot comparison
figure('Name', 'Euler Angles: Euler vs RK4');
subplot(3,1,1);
plot(time, euler_deg_euler(:,1), 'r', t_rk4, euler_deg_rk4(:,1), 'b');
ylabel('Yaw (deg)');
legend('Euler', 'RK4');

subplot(3,1,2);
plot(time, euler_deg_euler(:,2), 'r', t_rk4, euler_deg_rk4(:,2), 'b');
ylabel('Pitch (deg)');

subplot(3,1,3);
plot(time, euler_deg_euler(:,3), 'r', t_rk4, euler_deg_rk4(:,3), 'b');
ylabel('Roll (deg)');
xlabel('Time (s)');

% Differences between methods
difference = euler_deg_rk4 - euler_deg_euler(1:length(t_rk4), :);

figure('Name', 'Angle Differences: RK4 - Euler');
plot(t_rk4, difference(:,1), 'r', t_rk4, difference(:,2), 'g', t_rk4, difference(:,3), 'b');
xlabel('Time (s)');
ylabel('Angle Difference (deg)');
legend('Yaw', 'Pitch', 'Roll');
grid on;

% Vector norms at specific times
time_points = [2000, 4000, 6000];
norms = zeros(length(time_points), 2);

for i = 1:length(time_points)
    idx = time_points(i)/delta_t + 1;
    norms(i, 1) = norm(euler_deg_euler(idx, :)); % Euler method

```



```

        norms(i, 2) = norm(euler_deg_rk4(idx, :));    % RK4 method
end

% Display table
disp('Table 1: Euler angle vector norms');
disp(array2table(norms, 'VariableNames', {'Euler_Method', 'RK4_Method'}, ...
    'RowNames', {'2000s', '4000s', '6000s'}));

%% Section C: DCM and 3-1-3 Euler Angles
% Compute DCM for all time steps (using Euler method results)
DCM = zeros(3,3,N+1);
for i = 1:N+1
    DCM(:, :, i) = angle2dcm(euler_history(i,1), euler_history(i,2), euler_history(i,3))
end

% Display DCM at specific times
disp('Table 2: DCM at selected times');
for i = 1:length(time_points)
    idx = time_points(i)/delta_t + 1;
    fprintf('DCM at %ds:\n', time_points(i));
    disp(DCM(:, :, idx));
end

% Extract 3-1-3 Euler angles from DCM
euler_313 = zeros(N+1, 3);
for i = 1:N+1
    % Yaw ()
    euler_313(i,1) = atan2(DCM(1,3,i), DCM(2,3,i));
    % Pitch ()
    euler_313(i,2) = acos(DCM(3,3,i));
    % Roll ()
    euler_313(i,3) = atan2(DCM(3,1,i), -DCM(3,2,i));
end

% Convert to degrees and wrap to  $\pm 180^\circ$ 
euler_313_deg = mod(rad2deg(euler_313) + 180, 360) - 180;

% Plot 3-1-3 Euler angles
figure('Name', '3-1-3 Euler Angles from DCM');
plot(time, euler_313_deg(:,1), 'r', time, euler_313_deg(:,2), 'g', time, euler_313_deg(:,3), 'b');
xlabel('Time (s)');
ylabel('Angle (deg)');
legend('Yaw ()', 'Pitch ()', 'Roll ()');
grid on;

%% Section D: Comments (to be included in report)
% Differences between 3-2-1 (Section B) and 3-1-3 (Section C) sequences:
% 1. Rotation order is different (Z-Y-X vs Z-X-Z)

```

% 2. They don't represent the same angular state due to different rotation sequences
 % 3. Differences come from the mathematical representation of rotations
 % 4. 3-1-3 is better for spacecraft, while 3-2-1 is more intuitive for aircraft

%% Supporting Functions

```
function dydt = euler_kinematics(t, y, w_history, time)
    % Find nearest time index
    [~, idx] = min(abs(time - t));
    w = w_history(idx, :);

    psi = y(1);
    theta = y(2);
    phi = y(3);

    % Kinematic equation matrix (3-2-1 sequence)
    Kinematic_Matrix = (1/cos(theta)) * [...
        0, sin(phi), cos(phi);
        0, cos(phi)*cos(theta), -sin(phi)*cos(theta);
        cos(theta), sin(phi)*sin(theta), cos(phi)*sin(theta)];

    dydt = Kinematic_Matrix * w;
end
```